Foggy LiDAR 3D Detection Project (M.Tech, IIIT Gwalior)

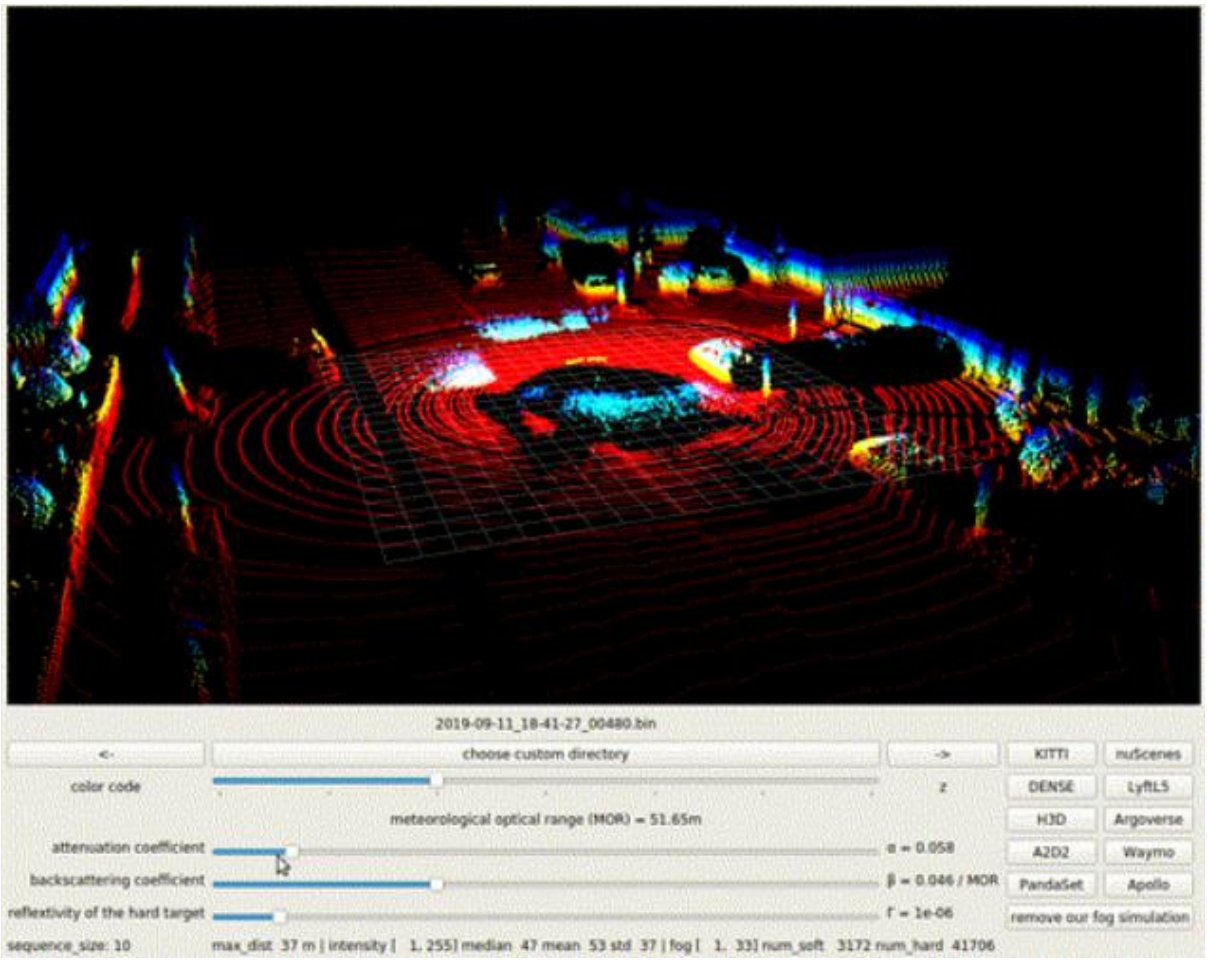
Figure: A LiDAR-equipped self-driving car in urban traffic. This project augments KITTI LiDAR data with simulated fog (using Mie scattering physics) to train robust 3D detectors. We create a large synthetic foggy dataset by varying fog density and particle noise, then train a state-of-the-art PointPillars network (via OpenPCDet) on both clear and foggy data. The goal is >77% 3D mAP (moderate, car class) on KITTI and **20–30% gains in foggy conditions** over baselines. Physics-based simulation for adverse weather has been shown to boost real-world performance.

Key Novelty:

- **Fog Simulation Pipeline:** We apply rigorous Mie-scattering models to inject realistic fog/aerosol effects into KITTI LiDAR point clouds, generating 10× more data.
- **Weather-Invariant Training:** Train on mixed clear+fog data (random fog density, particle dropout, geometric transforms) to learn features robust to poor visibility.
- **SOTA Target:** Leverage PointPillars (a leader on KITTI) and aim to exceed 77% AP (car, mod.) by full training and fine-tuning.

Model: PointPillars Pipeline

- *Example of a 3D detection workflow (e.g. PointPillars pipeline).* Usage of **PointPillars** because it efficiently encodes sparse 3D points into a 2D pseudo-image for CNN processing. Specifically, PointPillars voxelizes the ground plane into vertical columns (“pillars”) and applies a light PointNet encoder per pillar. The resulting feature map feeds a 2D CNN (multi-block backbone + SSD head) to predict 3D boxes. In practice, PointPillars achieves high throughput (~62 Hz on a 1080Ti) with strong accuracy (KITTI moderate BEV mAP ≈86.1% for cars), outperforming older voxel-based encoders. We adopt standard OpenPCDet/MMDetection3D tools and train with AdamW for 100 epochs. Training details: 0.16 m voxel size, batch 4–8 (Colab GPU), monitor Val mAP@0.7. Evaluation uses KITTI 3D metrics (AP for car/pedestrian/cyclist, easy/moderate/hard).

- 

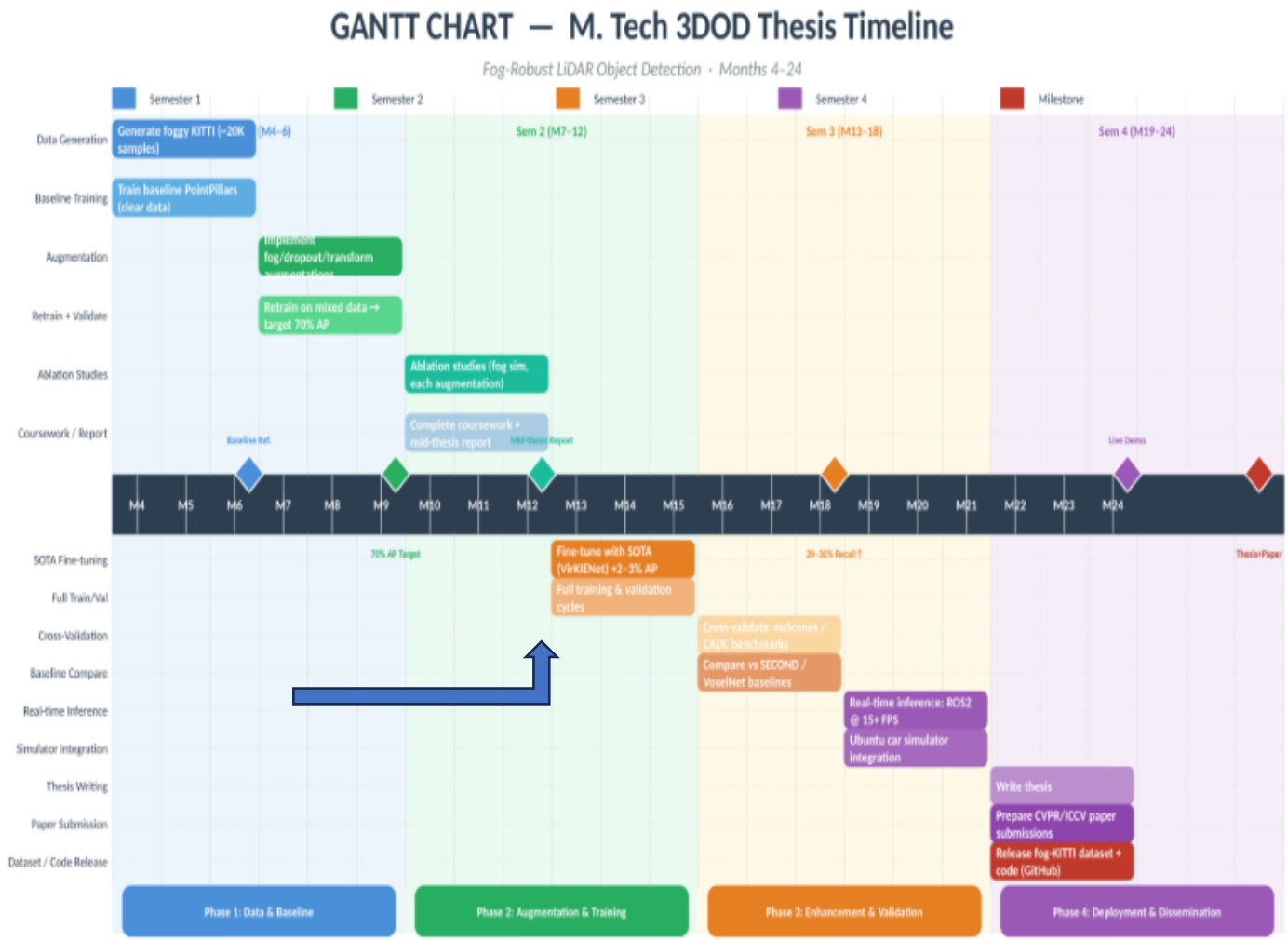
Data & Augmentation

We use the KITTI 3D dataset (7481 train + 7518 test frames). Our **augmentation pipeline** creates ~50K training samples by:

- **Fog Density Shift:** Randomly vary Meteorological Optical Range (visibility) from 100–1000 m using ray-based light attenuation.
- **Particle Dropout:** Randomly drop 10–50% of points to mimic aerosol backscatter noise.
- **Geometric Transforms:** Random rotations and scaling (Open3D) to increase diversity.

LiDAR point cloud with dense fog simulation (as in Martin Hahner Paper | |).

By simulating fog's effect on LiDAR returns (including multi-scattering), we generate realistic point clouds without new data collection



Training Workflow

0. **Where we are Currently: calculate attenuation [extinction] coefficient {qext} || (α), and backscattering coefficient {qb} || (β), separately for A) Fog medium, B) Aerosol Medium, and C) joint (Smog) Medium.**
1. **Preprocessing:** Voxelize points into 0.16 m pillars. Filter or augment points per above.
2. **Training:** Use PointPillars with AdamW (learning rate schedules from OpenPCDet). Train for 100 epochs, validate on held-out split of mixed data.
3. **Validation:** Compute KITTI-style AP (easy/mod/hard) on car, pedestrian, cyclist. Also evaluate specifically on a foggy validation split.
4. **Comparison:** Benchmark against a clear-weather baseline (no fog) and other SOTA models (e.g. SECOND, PV-RCNN) to quantify gains in fog.

The gantt, proposed is supposed to be followed. Key phases include:

- ***Months 1–3 (Sem 1):*** Setup environment (Colab/Anaconda, OpenPCDet, Open3D). Download KITTI; extend visualization tools. Begin coding fog simulation GUI. Also, we could work on 2D images (for comparing YOLO8 with other succesfull models (which are made for 3D)). Begin coding YOLO8
- ***Months 4–6 (Sem 1):*** Generate initial foggy KITTI (≈20K samples) [2D and 3D]. Train baseline PointPillars on clear data (70/15/15 split) to establish reference performance. And also train YOLO8 on 2D (corresponding ≈20K samples), whatever we will do in 3D with other models, similarly, will do parllely on 2D for YOLO8, to compare which is better
- ***Months 7–9 (Sem 2):*** Implement augmentations (fog, dropout, transforms) in data loader. Retrain PointPillars on mixed data. Target ~70% AP (car mod.) on fog hold-out. Same here on 2D correspondingly.
- ***Months 10–12 (Sem 2):*** Perform ablation studies (e.g. without fog sim) to quantify each augmentation's effect. Complete first-year coursework and mid-thesis report. Same studies will do corresponding to YOLO8 but on 2D dataset.
- ***Months 13–15 (Sem 3):*** Fine-tune with SOTA enhancements (e.g. ViKIENet tweaks) to boost AP by +2–3%. Run full training/validation cycles. Here will do for corresponding to YOLO8 related enhancements.
- ***Months 16–18 (Sem 3):*** Cross-validate on other foggy benchmarks (e.g. nuScenes/CADC) and compare to SECOND/VoxelNet baselines. Aim for 20–30% better recall under fog.
- ***Months 19–21 (Sem 4):*** Demonstrate real-time inference (ROS2 simulation at 15+ FPS). Integrate system in an Ubuntu-based car simulator for live tests.
- ***Months 22–24 (Sem 4)*****:** Write thesis; prepare CVPR/ICCV paper submissions. Release the fog-augmented KITTI dataset and code on GitHub.

Roughly:

raining Setup:

Pillar size 0.16 m, AdamW optimizer, 100 epochs, batch 4–8.

Evaluation:

KITTI metrics (3D AP at 0.7 IoU) for Car/Ped/Cyclist, plus dedicated foggy-scene benchmarks.

Goal: >77% 3D moderate-car mAP on KITTI (clear) and ~25% relative AP increase on foggy test sets.